MailoHLS: Multi-Adapter Structure-Aware Learning for Pareto-Driven HLS Pragma Optimization

MICRO 2026 Submission **#3267** – Confidential Draft – Do NOT Distribute!!

Abstract

High-Level Synthesis (HLS) enables rapid development of FPGA accelerators, yet achieving high-quality results (QoR) remains challenging due to the large and irregular design space induced by compiler directives (a.k.a *pragmas*). Selecting effective configurations requires reasoning over complex interactions between program structure, memory behavior, and often conflicting objectives such as latency and resource utilization. Prior model-driven approaches exhibit limited generalization across kernels and fail to capture higher-level optimization intent. Recently, Large Language Models (LLMs) capture code semantics and high-level intent, but their sequential representations hinder modeling of structural dependencies and global trade-offs, leading to suboptimal HLS designs.

We present MailoHLS, a hybrid framework that combines LLM-based semantic reasoning with GNN-based structural modeling for objective-aware directive optimization. By integrating structural embeddings via cross-attention and leveraging PEFT with objective-conditioned LoRA adapters and Pareto-driven optimization, MailoHLS enables joint reasoning over code semantics, structure, and design trade-offs. Across seen and unseen kernels, MailoHLS achieves up to 12.42× and 8.4× speedup (9.48× and 4.97× geometric mean) for latency optimization, consistently producing near-Pareto-optimal designs. On fully unseen applications, it reaches up to 10.2× speedup (6.58× geometric mean), outperforming high-end LLMs and prior approaches while narrowing the gap to the Pareto frontier.

Keywords

FPGAs, HLS, DSE, LLMs, LoRA, Structure-Aware Optimization, Objective-Aware Adaptation

1 Introduction

Hardware accelerators are increasingly deployed across cloud and edge environments, with field-programmable gate arrays (FPGAs) playing a key role due to their flexibility and energy efficiency. Cloud platforms such as Microsoft's Project Catapult [51], IBM's CloudFPGA [8], and Amazon EC2 F2 instances [2] highlight their adoption in large-scale, latency-critical services. At the edge, FPGAs are also widely used in domains such as networking [9] and robotics [64], where their deterministic execution, low latency, and reconfigurability suit real-time workloads.

Despite their advantages, designing efficient FPGA accelerators remains challenging. Achieving high performance typically requires low-level hardware design in Hardware Description Languages (HDL), which is time-consuming and demands significant expertise. High-Level Synthesis (HLS) addresses this by raising the abstraction level from HDL to C/C++-like specifications [16], designers guide the generated microarchitecture through compiler directives (a.k.a. *pragmas*) that control loop transformations, parallelism, and memory behavior. Consequently, achieving high-quality results (QoR), such as latency and resource utilization, critically depends on selecting effective combinations of these directives. However, directive effects are highly interdependent: decisions on loops or memory structures can impact other parts of the program in non-obvious ways. For example, memory partitioning directly constrains exploitable parallelism; insufficient partitioning may render aggressive loop unrolling infeasible due to bandwidth limitations. This leads to a vast design space, where each configuration must be evaluated through costly synthesis runs (often minutes to hours). The problem is further compounded by the need to balance multiple, often conflicting objectives, such as latency, resource utilization, and energy efficiency.

To tackle this challenge, prior research has introduced automated design space exploration (DSE) frameworks for high-level synthesis (HLS). Early efforts primarily relied on *synthesis-based* approaches [25, 26, 41, 57], which explore the configuration space by evaluating quality-of-result (QoR) metrics obtained from designs compiled with HLS tools. While these methods are generally framework-agnostic, they often suffer from slow convergence, as insights about the design space are gathered incrementally and each exploration step requires expensive synthesis runs. More recently, *model-driven* [18, 40, 42, 71] and *data-driven* [21–23, 29] approaches have been proposed to mitigate the cost of iterative synthesis. Model-driven techniques replace costly synthesis in the optimization loop with predictive models that estimate QoR metrics, whereas data-driven methods leverage knowledge from previously explored designs to guide optimization for new applications. However, many model-driven approaches remain application-specific, often requiring retraining for each new design. To address this limitation, graph neural networks (GNNs) have emerged as a promising direction [27, 58, 59, 61]. By representing programs as graphs, GNNs can effectively capture structural characteristics, such as control flow, data dependencies, and memory access patterns, enabling more generalizable and efficient design space exploration [44]. Yet, these approaches primarily focus on structural representations and often lack the ability to reason about higher-level semantics or optimization intent. As a result, their effectiveness can be limited when generalizing across diverse workloads or adapting to different optimization objectives.

In parallel, recent advances in Large Language Models (LLMs) demonstrate strong capabilities in code understanding and generation, motivating their application to hardware design automation [48]. State-of-the-art models, such as GPT-4 [47] and Claude [3], are already integrated into development environments (e.g., VS Code), enabling developers to interactively generate, refine, and optimize code within their workflows. However, in the context of HLS optimization, LLMs often struggle to produce efficient and reliable directive configurations, particularly for complex kernels, and may

MICRO 2026, Athens, Greece
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

even generate invalid or incorrectly placed pragmas. The reason is that LLMs primarily rely on sequential token representation and generation without explicit modeling of program structure or non-functional objectives. This makes it difficult to capture complex dependencies across loops, memory accesses, and hardware resources, as well as to reason about performance metrics such as latency and resource utilization. Moreover, recent LLM-based research works typically rely on structure-agnostic adaptation techniques, such as prompt engineering [30, 33], retrieval-based augmentation [13, 68], or indirect incorporation of structural information [50, 65]. These methods provide implicit guidance but do not explicitly model structural dependencies or hardware constraints. Hence, their ability to generate efficient and valid configurations remains limited.

In this paper, we propose MailoHLS, a hybrid approach for HLS optimization that combines *structural* and *semantic* reasoning. To capture structural information, MailoHLS constructs a directive-aware graph representation encoding control flow, data dependencies, and memory interactions. In parallel, it leverages an LLM backbone to model source-level semantics and optimization intent. These representations are fused through *cross-attention*, enabling joint reasoning over structural constraints and semantic context. Rather than relying on code generation, MailoHLS reformulates HLS optimization as a directive-level prediction problem, where the model assigns values to predefined directive placeholders. This design (i) constrains the search space to valid configurations, (ii) improves robustness by focusing on optimization decisions rather than syntax generation, and (iii) aligns with modern HLS workflows. Finally, MailoHLS incorporates objective-aware adaptation through lightweight LoRA adapters, enabling specialization across different design goals. More specifically, MailoHLS introduces:

- **Hybrid structural–semantic modeling for HLS optimization:** We propose MailoHLS, which combines directive-aware graph representations with LLM-based semantic reasoning via cross-attention, enabling joint reasoning over program structure and optimization decisions.
- **Directive-level formulation of HLS optimization:** Instead of relying on LLM-based code generation or modification, we reformulate HLS optimization as a structured prediction problem over directive placeholders, decoupling optimization decisions from code synthesis and constraining the search space to valid configurations.
- **Pareto-driven, objective-aware optimization:** We introduce a training framework based on Pareto-ranked design data and lightweight LoRA adapters, leveraging supervised fine-tuning and preference-based optimization to specialize the model across multiple objectives.

We evaluate MailoHLS on seen kernels, leave-one-family-out transfer, and fully unseen applications. On seen kernels, it achieves 9.48×, 7.95×, and 2.10× geometric-mean speedup for latency-, balanced-, and resource-oriented objectives, closely tracking—and often surpassing—Pareto reference points. Under the MachSuite holdout setting, it retains strong performance (4.97×, 5.21×, 2.45×), demonstrating robust generalization across unseen kernel families. On fully unseen applications, it achieves up to 6.58× speedup, outperforming high-end LLMs, which frequently produce invalid or suboptimal designs, while consistently generating Pareto-efficient and synthesizable configurations.

Listing 1: GEMV HLS kernel with different pragma configurations.

```
const float A[N][M];
#pragma HLS ARRAY_PARTITION variable=A cyclic factor=8 dim=2
const float x[M];
#pragma HLS ARRAY_PARTITION variable=x complete
float y[N];
#pragma HLS ARRAY_PARTITION variable=y complete
for (int i = 0; i < N; i++) {
#pragma HLS PIPELINE II=1
  float acc = 0;
  for (int j = 0; j < M; j++) {
    #pragma HLS UNROLL factor=8
    acc += A[i][j] * x[j];
  }
  y[i] = acc;
}
```

2 Background & Related Work

2.1 HLS Directive Optimization

HLS is an FPGA design methodology that translates high-level programming descriptions into hardware accelerators, reducing the need for manual HDL design [55]. By abstracting low-level implementation details, HLS allows designers to focus on functional behavior, while the compiler performs scheduling, resource allocation, and Register Transfer Level (RTL) generation [15]. To optimize performance and resource efficiency, developers use directives or *pragmas* to guide the compiler, enabling microarchitectural customization and exploration of design trade-offs. These directives shape key aspects of the design, such as parallelism, pipelining, and memory behavior, that directly impact QoR metrics, including latency and resource utilization [49].

Listing 1 shows the HLS code for a General Matrix–Vector Multiplication (GEMV) kernel. The first directive applies cyclic partitioning to array A across 8 memory banks along the second dimension, enabling concurrent column accesses and increasing memory bandwidth. The unrolling directive replicates the inner loop by a factor of 8, allowing multiple multiply–accumulate operations to execute in parallel and improving throughput. Each directive is associated with a specific source-code location (Action Point – AP), which affects the generated hardware. For example, loop-level directives are applied immediately after the corresponding loop declaration.

The effectiveness of HLS highly depends on selecting appropriate pragmas for each application. However, identifying optimal configurations for a given objective remains challenging [56], due to the large design space and diverse directive options [45]. For example, as shown in Listing 1, array partitioning supports multiple strategies (e.g., *block*, *cyclic*, *complete*) with tunable factors across dimensions. Achieving high performance requires coordinated use of multiple directives [22, 23], as poor choices can lead to inefficient resource utilization and underutilized parallelism, sometimes resulting in performance below that of CPUs [12].

Automating Directive Optimization: To alleviate manual tuning, prior work has explored automated design space exploration (DSE) for HLS directive optimization. However, this problem remains challenging due to the exponential growth of the design space

with the number of action points and directive options, the high cost of evaluating each configuration via full HLS synthesis, and the need to balance multiple, often conflicting objectives, such as latency or allocated resources on the device. Early *synthesis-based* methods [25, 26, 41, 72] treat the HLS tool as a black box, iteratively querying it to obtain QoR metrics. While general, they suffer from slow convergence due to expensive evaluations. To mitigate this, *data-driven* approaches [5, 21–24, 28, 29] reuse prior design knowledge to guide exploration, reducing the number of required synthesis runs. In parallel, early *model-driven* methods [18, 40, 42, 71] employed ANN and CNN models to predict QoR metrics from pre-synthesis features, avoiding full compilation. Despite these advances, existing approaches either rely on costly evaluations or depend on handcrafted features and limited generalization.

2.2 Graph Neural Networks

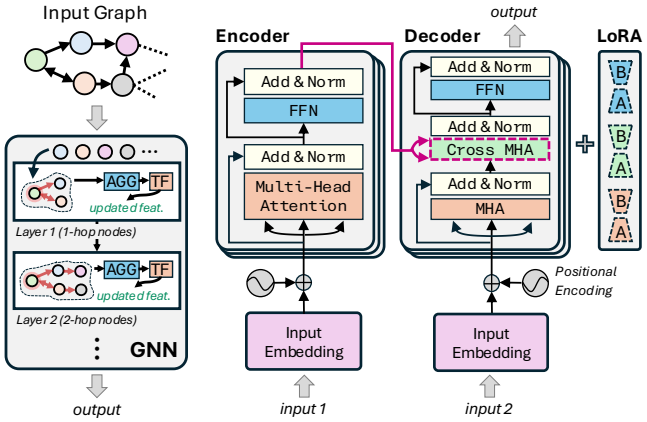
GNNs have been widely adopted as an alternative for HLS optimization to model and explore the large design space induced by pragma configurations [4, 6, 31, 39, 44, 58, 59, 61, 67]. As shown in Figure 1a, they learn structure-aware representations through iterative message passing, where each node aggregates information from its neighbors and applies a learnable transformation, capturing progressively broader dependencies. Thus, *GNNs learn structure-aware representations that capture graph topology and dependencies.*

GNNs align naturally with HLS, where kernels are represented as graphs (e.g., Control Data Flow Graphs – CDFGs) encoding control and data dependencies. However, QoR often depends on long-range interactions (e.g., between nested loop transformations and memory hierarchy), which are difficult to capture with standard graph representations and localized message passing. Modeling such dependencies requires deep GNNs, which can suffer from over-smoothing and training instability [1, 46, 69]. To address this, approaches such as HARP [59] augment graphs with *auxiliary nodes* that encode higher-level structure, shortening distances between dependent nodes and improving information propagation.

2.3 Transformer, Cross-Attention and LoRA

While GNNs effectively capture structural dependencies, they struggle to model code semantics and global optimization intent. Recent advances in LLMs have significantly improved their ability to reason about code and perform complex program transformations, driving growing interest in hardware design automation [13, 33, 43, 50, 52, 60]. At the same time, LLMs such as GPT-4 [47] and Claude [3] are already integrated into development environments (e.g., VSCode), enabling interactive code generation and refinement.

Transformers [62], the backbone of LLMs, learn context-aware semantic representations by modeling relationships across tokens through attention mechanisms. As shown in Figure 1b, they can be structured as encoder–decoder architectures, where attention (MHA) enables global information exchange, while feed-forward layers (FFN) apply learned token-level transformations. Cross-attention (cross MHA) further enables the integration of external representations, making it well-suited for fusing complementary modalities such as structural features. Thus, *Transformers learn context-aware semantic representations that capture relationships across tokens and long-range dependencies in the input.*



(a) GNN (b) Transformer and Low-Rank Adaptation

Figure 1: Overview of GNN and Transformer architectures.

Adapting large Transformer models to domain-specific tasks is computationally expensive due to their scale. Parameter-Efficient Fine-Tuning (PEFT) addresses this by updating only a small subset of parameters. Low-Rank Adaptation (LoRA) [34] injects lightweight trainable updates into selected layers, enabling efficient specialization. In HLS, such adapters can be conditioned on different optimization objectives, allowing the model to balance competing design goals while preserving general knowledge.

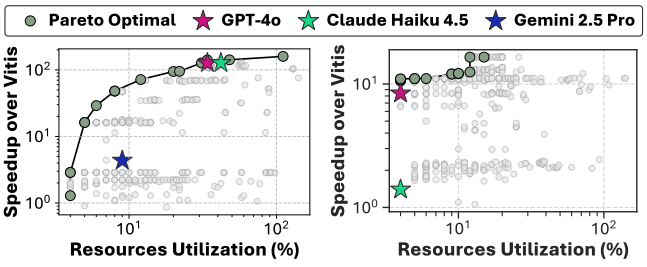
3 Motivation

3.1 Structural nature of HLS optimization

HLS optimization is inherently structural, as performance depends not only on selecting and placing directives, but also on their interactions, which collectively shape the generated hardware. Directives influence key aspects such as parallelism and memory access, and are tightly interdependent. For example, array partitioning determines available memory bandwidth, which constrains achievable parallelism through loop unrolling. In Listing 1, partitioning arrays A and x with a factor of 2 makes an inner loop unroll factor of 8 infeasible, as the memory system cannot supply operands fast enough to sustain parallel execution. Such imbalances lead to resource underutilization and degraded performance. Consequently, HLS optimization requires reasoning over a tightly coupled design space, where QoR is determined by the global hardware organization rather than isolated directive choices, motivating models that capture structural dependencies and their interactions.

3.2 Limitations of Decoder-Only LLMs for HLS

Modern large-scale LLMs, such as GPT-4o [47] and Claude Haiku 4.5 [3], follow a decoder-only architecture, modeling code as a sequence of tokens and capturing semantic relationships through autoregressive attention. While this enables strong capabilities in code understanding and generation [10], it inherently limits their ability to explicitly model structural dependencies in programs. However, as discussed in §3.1, in HLS performance optimization is fundamentally driven by structural transformations and their interactions. Although LLMs benefit from strong generalization



(a) GEMV (b) KNN

Figure 2: Positioning of decoder-only LLM generated designs compared to Pareto-optimal configurations from [24].

and extensive training on large code corpora, their token-centric reasoning often fails to capture hardware-level constraints and dependencies required for effective directive optimization.

To systematically investigate these limitations, we evaluate the capability of decoder-only LLMs to optimize HLS designs through prompt-based interaction. We consider three applications of increasing complexity: (i) the GEMV kernel of Listing 1, (ii) a K-Nearest Neighbors (KNN) implementation from the RodiniaHLS benchmark suite [14], and (iii) a Kalman filtering application sourced from GitHub. The LLMs are instructed to minimize latency under resource constraints on an AMD UltraScale+ ZCU104 platform (100 MHz), restricted to pragma insertion without modifying the source code. We evaluate GPT-4o, Claude Haiku 2.5, and Gemini 2.5 Pro. Generated designs are synthesized with Vitis HLS 2021.1 and assessed in terms of latency speedup and resource utilization against a baseline without directives (Vitis). To contextualize performance, we derive Pareto-optimal configurations using an NSGA-II-based design space exploration framework [24].

Figure 2 shows the Pareto frontiers obtained via NSGA-II and the LLM-generated designs for GEMV and KNN. For GEMV, the latency-efficient Pareto design achieves a 159.5× speedup over baseline Vitis, using 1%, 11%, 34%, and 65% of BRAM, DSP, FF, and LUT, respectively. In contrast, Gemini 2.5 Pro attains only 4.4×, failing to identify the y array as critical for throughput. As a result, no partitioning is applied, creating a memory bottleneck that prevents achieving an initiation interval (II) of 1 and limits the benefits of loop unrolling. In contrast, GPT-4o and Claude Haiku 2.5 correctly identify y as performance-critical and apply appropriate partitioning, enabling efficient memory access and sustained pipeline throughput, achieving up to 127.2× speedup within resource constraints.

Regarding KNN, the latency-efficient Pareto design achieves a 17× speedup over baseline Vitis, while utilizing only 1%, 1%, 2%, and 11% of BRAM, DSP, FF, and LUT resources, respectively. In contrast, Gemini 2.5 Pro generates directives that result in a compilation failure. Specifically, it introduces dataflow without properly restructuring memory accesses, causing multiple processes to read concurrently from the same global memory interface and violating the single-port constraints of bundled AXI interfaces in Vitis HLS. This behavior highlights a key limitation of general-purpose LLMs, which often fail to account for hardware-level constraints. A similar issue is observed with Claude Haiku 2.5: although loop unrolling is applied, the corresponding arrays are not partitioned, preventing true parallel memory access and limiting performance to 1.4× speedup. In contrast, GPT-4o adopts a more balanced strategy by combining pipelining (II=1), dataflow, and proper memory management, achieving a higher speedup of 8.4×, which nevertheless remains well below the optimal achievable performance.

Finally, for the Kalman filter, which is the most complex application considered, the latency-efficient Pareto design achieves a 35.3× speedup over baseline Vitis, with resource utilization of 92%, 7%, 7%, and 19% for BRAM, DSP, FF, and LUT, respectively. In this case, GPT-4o generates directives that lead to compilation failure due to misplaced array pragmas, applied before their corresponding declarations, whichi result in undefined variable errors. For Claude Haiku 2.5 and Gemini 2.5 Pro, both models apply overly aggressive directives, leading to BRAM over-utilization of up to 113%. *This behavior highlights the difficulty for general-purpose LLMs to consistently satisfy hardware resource constraints, often leading to impractical designs.*

3.3 Pitfalls of Structure-Agnostic Adaptation

High-level adaptation techniques, such as prompt engineering [48] and retrieval-augmented generation (RAG) [38], aim to guide LLMs through additional context and structured instructions. This concept has also been investigated in the HLS domain [68], where enriched prompts explicitly specify which directives should be applied and where they should be placed within the code. While these approaches improve guidance, they remain fundamentally structure-agnostic, relying on token-level context rather than explicit representations of program structure.

To assess their effectiveness, we design prompts that encode key HLS optimization strategies, including pipelining, loop unrolling, and array partitioning [29]. Beyond directive placement, the prompts also include tuning guidance (e.g., enforcing II=1 or aligning partitioning factors with loop unrolling). We evaluate this approach on KNN and Kalman filtering, representing the more complex applications. For KNN, enriched prompts improve performance across models. In particular, Gemini 2.5 Pro generates a valid design by correctly applying dataflow under AXI constraints, achieving an 8.4× speedup over baseline. GPT-4o and Claude Haiku 4.5 also benefit, reaching 6× and 15.6× speedups, respectively. However, this improvement does not generalize to more complex cases. In the Kalman filter, GPT-4o still produces invalid code due to incorrect pragma placement, while Claude Haiku 2.5 and Gemini 2.5 Pro exceed BRAM capacity (up to 113%), despite explicit guidance.

These results show that, *even with detailed and structured prompts, general-purpose LLMs struggle to consistently generate valid and efficient HLS designs.* While prompt engineering can guide the model toward better local decisions, it does not provide an explicit representation of program structure or hardware constraints. As a result, the model must infer complex structural relationships implicitly from token sequences, which is often unreliable, especially for long-range dependencies and tightly coupled optimizations. This limitation becomes more pronounced as application complexity increases, where multiple interacting transformations must be coordinated. Consequently, structure-agnostic adaptation remains insufficient for reliably navigating the HLS design space.

4 MailoHLS: Design Overview

To address these limitations, we propose MailoHLS, a Pareto-driven hybrid framework for HLS pragma optimization that explicitly integrates structural reasoning, semantic understanding and objective-aware adaptation. MailoHLS is designed around three principles:

(1) Joint structural & semantic modeling. MailoHLS combines graph-based structural representations with language-model reasoning to capture both the program organization and its high-level semantics. Unlike text-only approaches, it explicitly models interactions between loop transformations, memory accesses, and directive placement, enabling the model to reason about hardware-relevant constraints and their impact on QoR.

(2) Directive-Level Decision Abstraction. Instead of training the LLM to generate or modify code, MailoHLS formulates HLS optimization as a structured prediction problem over directive placeholders inserted at valid action points (e.g., loops and arrays). This abstraction decouples where directives apply from what values to assign, constraining the search space to valid configurations and improving robustness.

(3) Objective-Aware Optimization. MailoHLS supports multiple optimization goals (e.g., latency, area, or balanced trade-offs) through objective-conditioned specialization. By leveraging Pareto-ranked design data and preference-based alignment, the model learns to navigate trade-offs between competing objectives, enabling it to generate objective-aware configurations.

Figure 3 presents the end-to-end workflow of MailoHLS. Given an input C/C++ kernel (❶), the framework first identifies candidate optimization action points (e.g., loops and arrays) and augments them with parameterized directive placeholders (❷), which define the decision space for pragma selection. MailoHLS then constructs two complementary representations of the annotated program.

For the *structural representation* (❸), the code is translated into an extended hierarchical graph derived from LLVM IR, enriched with auxiliary nodes and directive-aware connections that capture loop hierarchy, memory behavior, and long-range dependencies. A GNN encoder processes this graph to produce node embeddings trained to reflect QoR-relevant properties (e.g., parallelism, data reuse, and resource pressure). These embeddings are aligned with directive placeholders and injected into the LLM through *cross-attention*.

For the *semantic representation* (❹), MailoHLS employs a decoder-only LLM to model code semantics and directive intent. To integrate structural context, the placeholder-aligned graph embeddings are injected into the LLM through cross-attention layers, applied selectively at directive prediction tokens. A gating mechanism regulates the contribution of structural and semantic signals, allowing the model to jointly reason about program logic and hardware constraints when predicting pragma values.

Finally, MailoHLS predicts the values of the directive placeholders (❺) by jointly leveraging semantic and structural information. To support different optimization goals, it enables objective-aware specialization through lightweight adapter modules (❻). Each adapter is trained for a specific objective using supervised fine-tuning followed by preference-based alignment on Pareto-ranked design pairs. At inference time, the appropriate adapter is activated based on the target objective, guiding the model toward configurations that reflect the desired trade-off.

5 MailoHLS: Detailed Design

5.1 Code Parsing & Placeholder Placement

To avoid syntactic inconsistencies (§3.2), MailoHLS operates over a structured formulation of the optimization problem. Given an input kernel ❶, MailoHLS first automatically parses the code to identify candidate action points where optimization pragmas can be applied, such as loop constructs and array definitions. These locations define the target design space and directly correspond to performance–resource trade-offs in HLS. Each action point is annotated with a directive placeholder token (e.g., <L1>, <L2>, <L3>) and every placeholder is associated with a predefined set of valid, yet unassigned, pragmas (❷). For example, for a loop construct, this set may include pipelining, loop unrolling, tiling and other transformations, as follows:

```
L4: for (int i = 0; i < N; i++) {
#pragma HLS pipeline II=auto{_PIPE_L4}
#pragma HLS unroll factor=auto{_UNROLL_L4}
...
```

This representation converts the kernel into a templated program where directive placement is fixed and only their values remain to be determined by the LLM backbone (§5.3). As a result, the model is restricted to operate over valid transformation points and predict directive values for each placeholder.

5.2 Structural Source Code Embeddings

After annotating the code, MailoHLS extracts *structural embeddings* by lowering the input kernel to an intermediate representation [37], building a hierarchical graph (❸b) and encoding it with a GNN (❸c).

5.2.1 LLVM Intermediate Representation. To enable consistent structural analysis across kernels, MailoHLS leverages an intermediate representation (IR) of the input code. IRs provide a normalized view of control flow, data dependencies, and memory operations, abstracting away syntactic variability in the code, such as variable naming and equivalent loop formulations [7, 59, 73]. Specifically, MailoHLS lowers the input kernel to LLVM IR [37], a low-level, Static Single Assignment (SSA)-based compiler representation. LLVM IR exposes constructs that directly relate to hardware behavior. For example, loop guards are represented through compare instructions (e.g., `icmp`), control transitions through branch instructions (e.g., `br`), and value and memory dependencies through explicit dataflow. A simplified example is shown below:

```
...
for.cond:
  %0 = load i32, i32* %i, align 4
  %cmp = icmp slt i32 %0, 32
  br i1 %cmp, label %for.body, label %for.end12
...
```

From this representation, MailoHLS identifies loop constructs by tracing compare instructions (e.g., `icmp`) and their associated branch targets (e.g., `br`), grouping them into loop-level regions that capture both control structure and associated computation. These structures expose hardware-relevant constraints such as pipelining feasibility, scheduling dependencies, and available parallelism. The resulting IR is then used as the basis for structured graph construction (§5.2.2) and subsequent GNN-based encoding (§5.2.3) to produce structural embeddings.

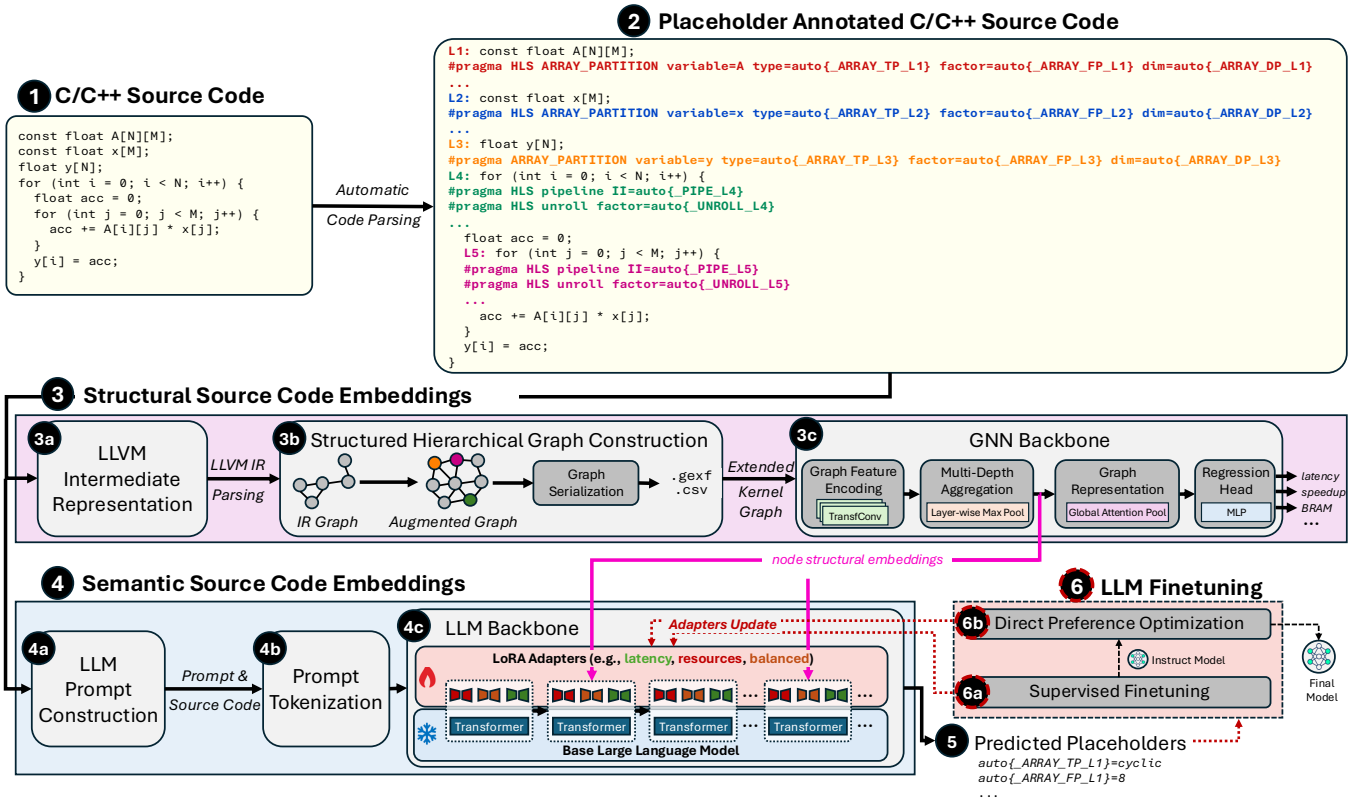


Figure 3: MailoHLS Design Overview. Given a C/C++ kernel, MailoHLS inserts directive placeholders and encodes structural and semantic embeddings via a GNN and an LLM, fused through cross-attention, with objective-specific LoRA adapters. Cold (❄) components denote frozen modules, while hot (🔥) components denote trainable parts.

5.2.2 Structured Hierarchical Graph Construction. Given the LLVM IR, MailoHLS constructs a ProGraML [17]-style graph representation, which encodes instructions, values, and control/data dependencies as a unified, SSA-based graph, that can be directly consumed by graph neural networks. However, directly applying such a representation to HLS optimization presents two key challenges. First, directive positions are not encoded in the IR, and therefore optimization pragmas that control parallelism and resource utilization are not explicitly represented. Second, the graph may exhibit long-range dependencies, where decisions at one loop level affect scheduling, pipelining, and resource sharing across distant regions, limiting the effectiveness of standard GNN models (§2.2).

To address these challenges, MailoHLS adopts a hierarchical graph construction similar to HARP [59], extending the base ProGraML graph with directive-aware and hierarchical augmentations tailored for HLS. The construction proceeds in multiple stages. Starting from the ProGraML graph, MailoHLS identifies directive placeholder locations and augments the graph with additional nodes and edges to encode both optimization decisions and structural hierarchy. Similar to HARP, MailoHLS introduces two types of nodes: *pragma* nodes and *auxiliary* nodes. Pragma nodes encode optimization directives, while auxiliary nodes capture hierarchical structure and connect distant regions of the graph. *Loop-related pragma nodes* (e.g., pipelining or unrolling) are attached to loop anchors in the IR, identified through compare instructions (e.g., icmp) that define loop boundaries. Beyond loop-centric modeling, MailoHLS extends HARP's representation with explicit support for shared data structures. *Array-related pragma nodes* (e.g., partitioning) are identified based on variable names and connected to IR nodes corresponding to array declarations and uses, with preference for declaration sites. This enables the graph to explicitly capture which compute regions interact through the same data structure.

Auxiliary nodes are then introduced to capture hierarchical scope. *Loop-level auxiliary nodes* group operations within the same loop region, while *data-centric auxiliary nodes* group accesses to the same array. These nodes are connected to their associated IR nodes and pragma nodes, and are further interconnected to form a hierarchical skeleton that reduces long-range dependencies. This construction exposes both control-flow and data-sharing relationships to the GNN. In particular, shared data structures introduce implicit coupling between distant compute regions, enabling the model to capture memory-related effects such as bandwidth constraints, access parallelism, and contention.

Figure 4 shows the constructed graph for the GEMV kernel in Listing 1. Gray nodes represent the graph obtained directly from LLVM IR, while colored nodes represent additional auxiliary and pragma nodes. As shown, *pragma nodes* encode optimization directives and are grouped into data-centric nodes (blue/red), associated

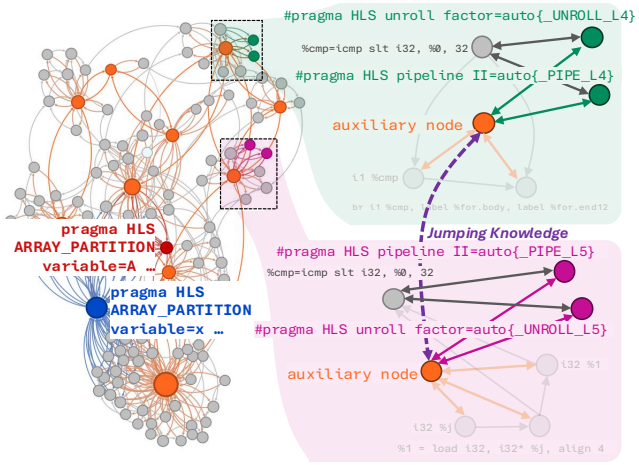


Figure 4: Graph representation of the GEMV kernel. Gray nodes represent LLVM IR operations, while colored ones show auxiliary and pragma nodes.

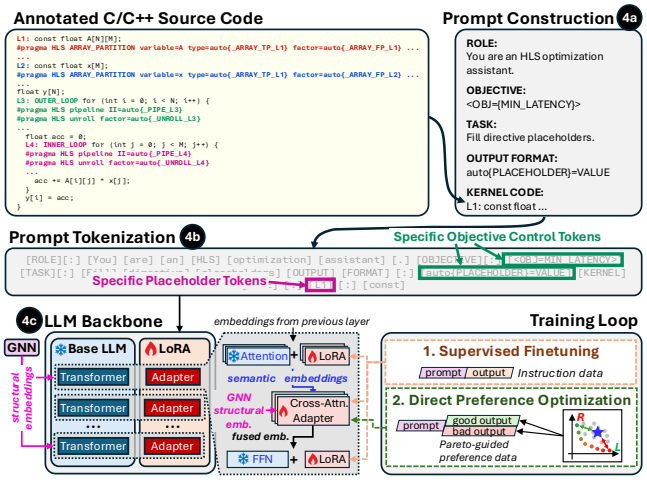


Figure 5: LLM backbone training pipeline with SFT, Pareto-guided preference optimization, and LoRA adapters.

with array transformations, and control-centric nodes (green/magenta), associated with loop transformations. Each pragma node is connected both to the corresponding LLVM IR instruction (e.g., `%cmp` defining loop boundaries) and to its associated auxiliary node, capturing both control-flow context and data dependencies between computation and memory accesses, while enabling a direct alignment with directive placeholder prediction. This maps directive choices in the local computation while propagating their impact across higher-level program structure. *Auxiliary nodes* (orange) capture higher-level structure and connect distant regions of the graph. Loop-level auxiliary nodes link control-flow constructs across nested loops, while data-centric auxiliary nodes aggregate accesses to shared arrays, strengthening dependencies between computation and memory behavior.

5.2.3 GNN Backbone. MailoHLS uses a separately trained GNN to generate structural embeddings of the input kernel. Starting from the extended hierarchical graph (§5.2.2), the GNN applies edge-aware TransformerConv message passing to encode control-flow, data dependencies, loop structure, and memory interactions into node-level representations. To capture both local and long-range dependencies, we employ multi-depth aggregation via Jumping Knowledge [70], which combines node embeddings from multiple GNN layers, effectively aggregating information across different hop distances. The resulting node embeddings are further pooled using global attention to obtain a graph-level representation. A regression head is trained on top of these embeddings to predict HLS QoR targets (e.g., latency and resource utilization), making the GNN an independent structural surrogate prior to any LLM-based fine-tuning. Unlike prior approaches such as HARP, MailoHLS does not use the GNN solely as a predictor. Instead, it reuses the learned embeddings before the regression head as structural inputs for the LLM (§5.2.2). n particular, the final node embeddings are extracted and converted into placeholder-aligned memory slots, which are later consumed by the LLM cross-attention layers.

5.3 Semantic Embeddings & Structural Fusion

The core of MailoHLS is its LLM backbone, which extracts *semantic embeddings* from the source code and integrates structural embeddings from the GNN to guide directive prediction. Semantic embeddings capture higher-level program properties (e.g., access patterns and loop roles) that are not explicitly represented in the structural graph but are critical for optimization decisions. As shown in Figure 5, the annotated kernel is transformed into a structured prompt (4a), tokenized (4b) and processed by the LLM (4c). Structural embeddings are incorporated through *cross-attention layers* injected at specific blocks in the LLM. To support multiple optimization goals, MailoHLS adopts a multi-adapter design, where each objective is handled by a dedicated LoRA adapter.

5.3.1 LLM Prompt Construction & Tokenization. The input source code is transformed into a structured prompt that constrains the task to directive prediction [54]. As shown at the top of Figure 5, the prompt includes a role description, a target objective, a task specification, and the expected output format. To explicitly encode this information, MailoHLS augments the tokenizer with *objective control* tokens (e.g., `<OBJ=MIN_LATENCY>`) and *placeholder* tokens. Directive sites are represented using placeholders (e.g., `<Lk>`), while directive assignments follow a fixed schema (e.g., `auto{PLACEHOLDER}=VALUE`), ensuring directive locations and objectives remain atomic during tokenization. This design enables the LLM to associate each directive decision with both its program context and optimization objective. In particular, placeholder tokens act as anchors that attend to (i) surrounding code regions (e.g., loop structure, memory accesses) and (ii) the objective control token, allowing each prediction to be conditioned on both structural context and the desired goal. As a result, the model learns a mapping from {*program context*, *objective*} to directive values, improving conditioning and task alignment [36, 66]. Since the directive schema is fixed, the model predicts only directive values rather than their placement, reducing the output space and improving reliability.

5.3.2 LLM Backbone & Structural Embeddings Fusion. MailoHLS' LLM backbone is based on a decoder-only model (e.g., DeepSeek-Coder [32]), which encodes the semantic context of the program and predicts directive values based on the input code and the target objective. However, as discussed in §3.2, semantic information alone is insufficient for HLS, as directive decisions depend on structural properties such as loop-carried dependencies, available parallelism, and memory access patterns. Still, standard decoder-only models do not natively support attending to external representations.

To incorporate structural information, MailoHLS augments the backbone with *cross-attention layers* that fuse GNN-derived embeddings into the token stream. As shown in Fig. 5, these layers are inserted between the self-attention module and feed-forward sublayers of selected transformer blocks, enabling structural information to be injected after semantic context aggregation and before value transformation (c.f. §2.3). To balance expressiveness and efficiency, cross-attention is applied periodically every n layers (e.g., every 2–4 layers), rather than in every Tranformer block.

The structural context is derived from the GNN as a set of node embeddings, which are organized into a fixed-size structural memory. During fusion, token embeddings act as queries (Q), while structural embeddings serve as keys (K) and values (V). Crucially, alignment between tokens and structure is achieved through the directive-level abstraction: target-anchor tokens (e.g., <Lk>) act as routing points that attend to corresponding structural elements associated with each directive site (e.g., loop nests, memory objects). Moreover, cross-attention is selectively applied only to target tokens corresponding to directive predictions, rather than uniformly across all tokens. This ensures that structural information directly influences optimization decisions, while preserving the stability of the fixed output schema. Finally, the cross-attention outputs are integrated through a gated mechanism, allowing the model to modulate the contribution of structural versus semantic signals. This fusion enables pragma predictions to jointly account for program semantics and structural dependencies, capturing constraints that affect pipelining, scheduling, and memory behavior.

LLM backbone sensitivity: To assess the impact of backbone capacity on final HLS quality, we compare MailoHLS with DeepSeek-Coder-1.3B and DeepSeek-Coder-7B as the semantic backbone. As shown in Figure 6, moving from the 1.3B to the 7B backbone increases the average achieved speedup from 4.13x to 4.89x, while the average resource footprint (BRAM, DSP, FF and LUT) decreases from 93% to 75%. This result indicates that increasing backbone capacity provides a modest QoR benefit within the same MailoHLS pipeline. The improvement is not dramatic, which suggests that the gains of MailoHLS do not depend solely on scaling the base LLM, but arise from the robustness of the overall framework, namely the combination of semantic reasoning, structural fusion through cross-attention, and objective-aware adaptation.

Cross-attention placement: We further study the effect of structural-fusion density by varying the insertion interval of cross-attention layers within the LLM. For the 32-layer DeepSeek-Coder-7B backbone, inserting cross-attention every 8 layers corresponds to 4 fusion points, while every 16 layers corresponds to only 2. Denser fusion improves the average speedup from 3.58x to 4.89x, at the cost of a higher total resource footprint from 68.0% to 93.3%. More frequent cross-attention strengthens the influence of structural memory and sharpens objective-specific behavior, especially for aggressive latency-oriented designs. Conversely, with fewer insertion points, MailoHLS remains effective but behaves more conservatively, resembling the simpler SFT-only regime more closely than the fully structure-aligned model. In the remainder, we therefore use the every-8-layer configuration as the default setting.

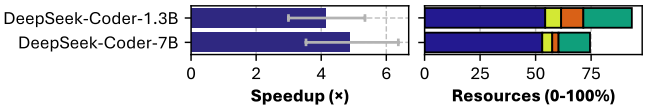


Figure 6: MailoHLS' sensitivity to employed LLM backbone.

5.3.3 Parameter-Efficient Objective-Aware Finetuning. MailoHLS adopts a parameter-efficient finetuning (PEFT) approach. Instead of updating all parameters, it employs LoRA [34] to introduce a small number of trainable matrices, enabling efficient specialization while preserving the pretrained semantic capabilities of the backbone. As shown in Figure 5, LoRA modules are applied to both attention and feed-forward layers, allowing the model to focus on directive placeholders, objective tokens, and relevant code regions during directive prediction.

To support multiple optimization goals (e.g., latency, power, resource usage), MailoHLS adopts a multi-adapter design, where each objective is handled by a dedicated LoRA adapter. At runtime, the appropriate adapter is selected based on the target goal [63]. Different objectives correspond to distinct trade-offs in the design space and often require conflicting directive configurations. For instance, minimizing latency increases parallelism and resource usage, while optimizing for power or area favors more constrained configurations. Thus, isolating adaptation per objective enables specialization across different regions of the Pareto frontier.

The training of each adapter is performed in three stages to decouple *i)* semantic alignment, *ii)* structural alignment, and *iii)* objective-aware preference modeling. In the first stage, supervised fine-tuning (SFT) [11] is applied to the LoRA matrices, training the model to imitate directive assignments from labeled examples. Training data consists of directive configurations evaluated through HLS, where each sample includes an annotated kernel and a directive assignment corresponding to a valid design point. This stage enables the model to learn a semantic mapping from code and objectives to feasible pragma configurations. In the second stage, LoRA adapters are frozen and SFT is applied only to the cross-attention layers, aligning structural embeddings from the GNN with the LLM representations. This stage enables the model to incorporate hardware-relevant constraints such as dependencies, parallelism, and memory behavior into its predictions without altering the learned semantic mapping. In the final stage, Direct Preference Optimization (DPO) [53] is applied to the cross-attention layers, learning to prefer better configurations over worse ones under the objective associated with the active adapter. Preference pairs are constructed from Pareto-optimal design points by selecting configurations that achieve better trade-offs under the target objective and contrasting them with dominated or inferior alternatives. This stage enables objective-aware learning, guiding the model toward directive choices closer to the desired region of the Pareto frontier.

6 Evaluation Setup

6.1 Training Dataset

For training and evaluation, we leverage the GNΩSIS dataset [24], one of the largest publicly available collections of HLS design points. By applying a diverse set of HLS directives, it captures both latency and resource utilization, resulting in a rich design space. However, GNΩSIS does not provide power measurements, and thus power is not considered as an optimization objective in this work. Overall, it comprises nearly 219K design points across two FPGA architectures and three target clock frequencies. We focus on the subset corresponding to the UltraScale+ ZCU104 platform at 100 MHz, which includes approximately 26,500 design points.

6.2 Training Parameters

We train one LoRA adapter per optimization objective using the three-stage pipeline described in §5.3.3. All adapters share a 4-bit quantized decoder-only backbone (NF4, bfloat16), namely Deepseek Coder 7B, with PagedAdamW8bit [20], cosine scheduling, and 3% warmup; gradient checkpointing supports 4K context prompts.

► **Stage 1 (Semantic Alignment):** We apply SFT on objective-conditioned directive completion, with cross-attention disabled. LoRA ($r = 8, \alpha = 16$) is applied to the LLM and trained for 4 epochs with a learning rate of 5×10^{-5}. We retain the top-6 design points per (kernel, objective) with score-based weighting.

► **Stage 2 (Structural Alignment):** Cross-attention is enabled to inject GNN embeddings, while LoRA and embeddings are frozen. Cross-attention is inserted every 8 layers (4 heads, dim 64), with structural memory of 64 slots (dim 32). Only cross-attention and gating parameters are trained for 4 epochs (lr=10^{-4} and 2×10^{-4}).

► **Stage 3 (DPO):** We apply DPO on Pareto-ranked preference pairs. LoRA remains frozen; only cross-attention and gating are updated (lr=10^{-5} and 2×10^{-5} for 3 epochs).

MailoHLS is publicly available as an open-source project[1].

6.3 Baselines and State-of-the-Art

We evaluate MailoHLS against a diverse set of baselines. As a reference, we consider the original source code without any HLS directives, as generated by VitisHLS. Furthermore, for each application in the GNΩSIS dataset [24], we extract the Pareto frontier and identify three representative design points: (a) the latency-optimal, (b) the resource-optimal, and (c) the Pareto knee, which captures a balanced trade-off between performance and resource utilization. These points serve as strong baselines for evaluating MailoHLS across different optimization objectives. We also compare MailoHLS against CollectiveHLS [23], an open SotA data-driven approach that leverages the GNΩSIS dataset to efficiently propose high-quality HLS directives. In addition, we benchmark against general-purpose large language models, including GPT-4o [47], Claude Haiku 4.5 [3], and Gemini 2.5 Pro [35], which are prompted to generate directive configurations. Finally, we include a comparison with LIFT [50], a SotA LLM-based framework for HLS optimization. LIFT integrates LLMs with GNN-based representations to capture both the sequential and structural characteristics of code, enabling the generation of performance-critical directives.

[1] Omitted for blind review

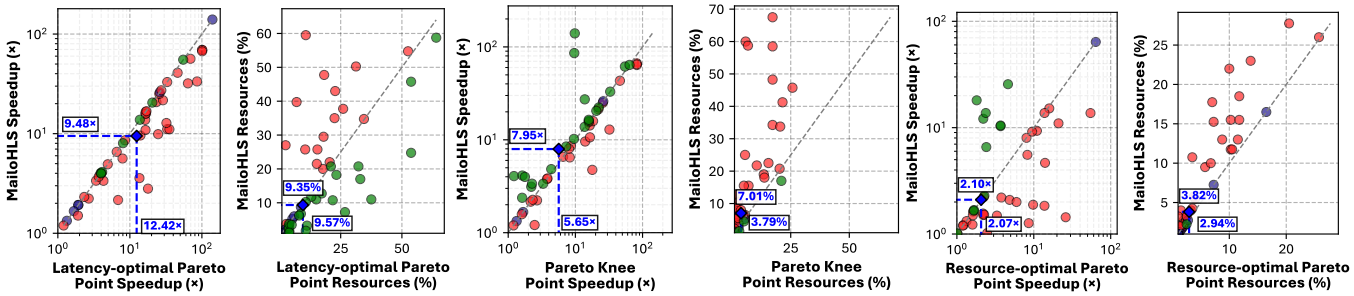
7 Experimental Results

7.1 MailoHLS Evaluation

7.1.1 Evaluation on seen kernels. We partition the dataset into 80%, 10%, and 10% splits for training, validation, and testing. We train MailoHLS to generate directive configurations conditioned on different optimization objectives, namely latency-optimized, resource-optimized, and balanced designs. The generated designs are synthesized using Vitis HLS 2021.1 to obtain latency and resource utilization metrics, which are then compared against the corresponding Pareto reference points (latency-optimal, Pareto knee, and resource-optimal) derived directly from GNΩSIS dataset. Figure 7 shows, for each objective, the speedup over Vitis and resource utilization relative to the Pareto reference. Diagonal points indicate optimal designs, while deviations reflect latency–resource trade-offs. Green points denote cases where MailoHLS dominates the reference (higher speedup and/or lower resources), highlighting the incompleteness of the GNΩSIS Pareto set, whereas red points indicate under-performance.

For the latency-optimized adapter, MailoHLS achieves a geometric mean speedup of 9.48× compared to 12.42× for the corresponding Pareto reference (0.76× relative), while maintaining slightly lower average resource utilization (9.35% vs. 9.75%). Most design points lie close to the diagonal, indicating that MailoHLS consistently approximates the target region of the design space. In several cases, the generated configurations dominate the reference points, demonstrating the ability of MailoHLS to uncover high-quality designs beyond those identified by the GNΩSIS exploration. For the balanced-optimized adapter, MailoHLS attains a geometric mean speedup of 7.95×, exceeding the Pareto knee reference of 5.65×, at the cost of higher average resource utilization (7.01% vs. 3.79%). Finally, for the resource-optimized adapter, MailoHLS achieves a geometric mean speedup of 2.10×, comparable to the reference value of 2.07×, with a marginal increase in resource utilization (3.82% vs. 2.94%). In this case, the distribution of points is more dispersed, due to the discrete and non-smooth nature of resource metrics in HLS, which makes precise control of utilization more challenging than latency-driven optimization.

To better understand the sources of design quality, we analyze the directives generated by MailoHLS across the different optimization objectives. For the seen kernels, the latency-optimized adapter applies `#pragma HLS PIPELINE II=1` to 34.8% of loop structures, compared to 31.9% for the balanced adapter and only 15.9% for the resource-optimized adapter. A similar trend is observed in loop unrolling. Aggressive unrolling factors are employed in 17.4% of latency-oriented loops and 11.6% of balanced loops, while they are not used in the resource-oriented setting. This behavior reflects the model's tendency to exploit parallelism more aggressively when optimizing for performance. The same pattern extends to memory optimizations. Large array partitioning factors are predicted for 7.1% of arrays in the latency adapter, compared to 3.6% and 1.8% for the balanced and resource-optimized adapters, respectively. In contrast, the choice of partitioning type (e.g., cyclic or block) remains relatively consistent across objectives. This suggests that objective-specific specialization is primarily achieved through varying the intensity of loop-level parallelism and memory banking, rather than through selecting different classes of directives.



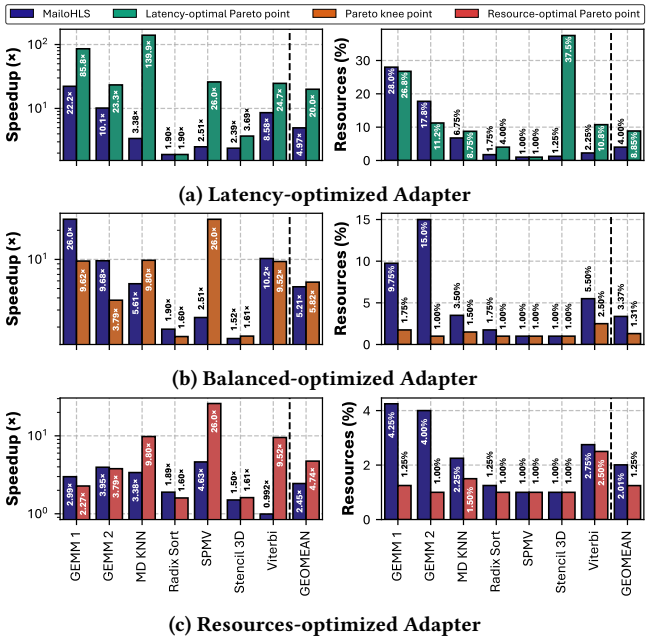
(a) Latency-optimized Adapter (b) Balanced-optimized Adapter (c) Resources-optimized Adapter

Figure 7: MailoHLS's evaluation under an 80/10/10 dataset split, where designs generated by the latency-, balanced-, and resource-optimized adapters are compared against GNΩSIS Pareto reference points in terms of speedup and resource utilization.

7.1.2 Evaluation on unseen kernels. To evaluate generalization, we adopt a leave-one-family-out setting in which the entire MachSuite kernels family is excluded from training and used solely for testing. Figure 8 reports the achieved speedup over Vitis alongside total resource utilization normalized to 0–100% for the three adapters. The latency-optimized adapter achieves a geometric mean speedup of 4.97×, below the 20× Pareto reference, while using significantly fewer resources (4% vs. 8.85%). The balanced adapter achieves a geometric mean speedup of 5.21×, closely approaching the Pareto baseline of 5.82×, with an average resource utilization of 3.37% compared to 1.31%. As expected, it occupies the intermediate region of the latency–resource trade-off space. Notably, it often proves more robust, in some cases matching or surpassing the latency-optimized adapter while avoiding higher resource overhead. The resource-optimized adapter achieves a geometric mean speedup of 2.45× compared to 4.74× for the Pareto baseline, with an average resource utilization of 2.01% versus 1.25%. While a performance gap remains, the results demonstrate that MailoHLS retains effective control over resource usage even under distribution shift, albeit with reduced precision relative to seen kernels.

These results further reveal that the three adapters learn and transfer distinct optimization policies. The latency adapter favors aggressive pipelining, loop-level parallelism, and memory partitioning when exploitable concurrency is inferred. In contrast, the balanced adapter exhibits greater robustness by avoiding extreme transformations, achieving competitive performance while maintaining moderate resource usage. The resource-oriented adapter deliberately constrains hardware replication and memory pressure, resulting in lower area at the cost of underutilized parallelism in compute-bound kernels. Finally, the larger gap to the Pareto reference in the unseen setting reflects the inherent difficulty of transferring hardware-aware optimization strategies (e.g., scheduling, parallelism, and memory access optimization). Nevertheless, MailoHLS consistently outperforms the directive-free baseline and remains well aligned with the target objective. This indicates that MailoHLS captures transferable micro-architectural principles, enabling effective generalization to unseen kernels without retraining or per-kernel exploration.

7.1.3 Ablation Study. To isolate the contribution of each training stage, we conduct an ablation study on the Kalman kernel, one of the most structurally demanding applications in our evaluation due



(a) Latency-optimized Adapter

(b) Balanced-optimized Adapter

(c) Resources-optimized Adapter

Figure 8: Speedup and Resources on Unseen Kernel Suite.

to its deeply nested loops, large on-chip buffers, and tight interplay between loop-level parallelism and memory partitioning. Figure 9 illustrates the achieved speedup over Vitis alongside resource utilization across the different stages of the training pipeline for the three objective-specific adapters.

Objective-conditioned SFT (*Step 1*) on the LoRA adapters produces only modest improvements, achieving speedups of 2×, 1.7×, and 1.2× for the latency, balanced, and resource-oriented adapters, respectively. The latency and balanced adapters over-utilize BRAM, while only the resource-oriented adapter respects FPGA constraints, indicating that semantic code understanding alone is insufficient to fully capture the structural limits required for aggressive pipelining, unrolling, and memory partitioning. The largest gains occur when GNN-derived structural information is incorporated through cross-attention (*Step 2*). At this stage, the latency, balanced, and

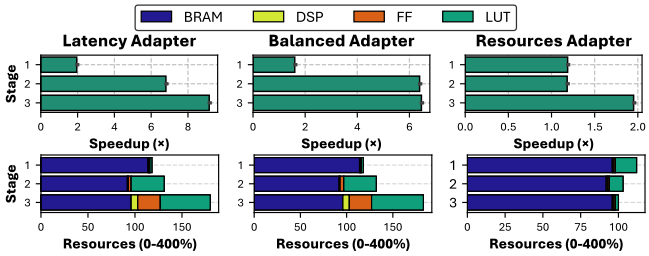


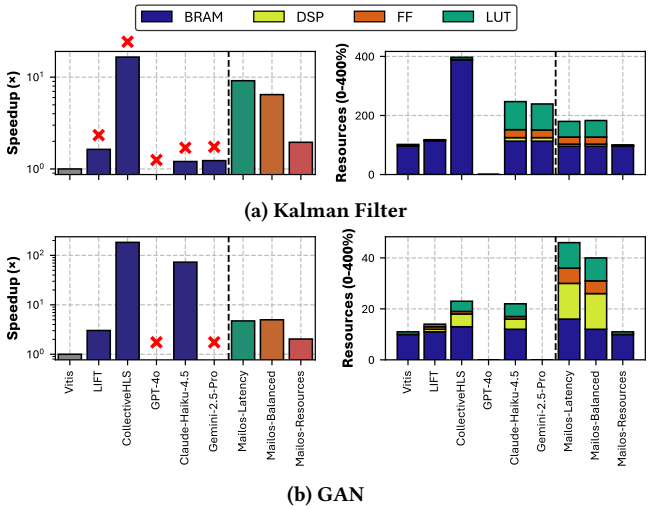
Figure 9: Impact of MailoHLS' training stages.

resource adapters achieve 6.3×, 6.2×, and 1.15× speedups, respectively. Exposure to loop hierarchy, dependency structure, and memory interactions enables the model to apply far more effective micro-architectural transformations. Although these improvements come with a slight increase in hardware usage, both the latency and balanced adapters now produce BRAM-compliant designs, highlighting the critical role of structural awareness. Finally, DPO (*Step 3*) aligns the generated designs with the target objectives, achieving speedups of 8.6×, 6.2×, and 1.9× for the latency-, balanced-, and resource-oriented adapters. For the latency adapter, DPO further boosts performance by steering designs toward higher-throughput configurations. The balanced adapter preserves most of the Stage 2 gains while controlling resource growth, consistent with its knee-region behavior. For the resource adapter, DPO enforces conservative resource usage while improving speedup relative to earlier stages. Overall, DPO effectively guides the final designs toward the intended Pareto-optimal trade-offs for each objective.

7.2 Comparison with State-of-the-Art

In the final stage of our analysis, we evaluate MailoHLS against prior SotA HLS optimization methods as well as high-end LLMs (c.f. 6.3). We use as driver applications the Kalman Filter and GAN [19], which stress complementary aspects of HLS optimization. The Kalman Filter kernel is memory-bound, with performance primarily limited by data movement, memory access patterns, and on-chip storage efficiency, whereas the GAN kernel is compute-bound, driven by arithmetic intensity, parallel execution of operations, and the ability to fully utilize available compute resources.

Figure 10 shows the respective results. For Kalman, MailoHLS delivers the strongest combination of design validity, quality of results, and alignment with optimization objectives. It produces valid designs achieving speedups of 8.6×, 6.2×, and 1.9× for the latency-, balanced-, and resource-optimized targets, respectively, all while respecting the FPGA's resource limits. In contrast, general-purpose LLMs prove unreliable: GPT-4o fails to generate any design, while Claude Haiku 4.5 and Gemini 2.5 Pro produce configurations that exceed FPGA capacity, reflecting their inability to handle the complexity of memory-bound applications. CollectiveHLS achieves high speedups on Kalman but requires excessive resources, rendering the designs non-feasible. LIFT, though structurally aware and closer to feasible designs, still over-utilizes BRAM due to incomplete handling of array partitioning directives. For the GAN application, which is MAC-intensive, highly aggressive loop parallelism can generate large speedups if activation buffers are sufficiently partitioned



(a) Kalman Filter

(b) GAN

Figure 10: Comparison with state-of-the-art on a memory- (Kalman) and a compute-bound (GAN) application.

to sustain bandwidth. CollectiveHLS and Claude Haiku 4.5 achieve remarkable speedups of 186.8× and 72.9×, respectively, while remaining within FPGA resource limits, showing that GAN favors extreme throughput-oriented transformations. While MailoHLS does not reach these extremes, it is the only method to produce three explicit objective-conditioned operating points from a single framework. It achieves 4.7×, 4.9×, and 2× speedups for the latency-, balanced-, and resource-oriented targets, all within the FPGA's capacity. The performance gap with CollectiveHLS and Claude Haiku 4.5 can be attributed to differing optimization strategies. In this case, MailoHLS favors explicit loop unrolling and array partitioning, whereas the competing methods rely more heavily on pipelining. In particular, applying pipelining at outer loops can implicitly trigger full unrolling of inner loops, significantly increasing parallelism and yielding higher speedups. Such aggressive transformations are not adopted by MailoHLS, as they risk violating resource constraints and leading to non-feasible designs. Instead, MailoHLS adopts a more conservative strategy, guided by feasibility considerations learned from prior design points. Last, MailoHLS outperforms LIFT, which achieves only a 3.1× speedup, particularly for the latency- and balanced-oriented designs, demonstrating its ability to generate efficient, well-balanced configurations across objectives.

8 Conclusion

We present MailoHLS, a unified approach for objective-aware HLS directive optimization that combines semantic reasoning with GNN-based structural modeling. By integrating structural embeddings via cross-attention and leveraging PEFT with objective-conditioned LoRA adapters and Pareto-driven optimization, MailoHLS enables joint reasoning over code semantics, structure, and design trade-offs. Across seen and unseen kernels, it achieves up to 12.42× and 8.4× speedup (9.48× and 4.97× geometric mean), consistently producing near-Pareto-optimal designs. On unseen applications, it reaches up to 10.2× (6.58× geometric mean), outperforming prior methods and general-purpose LLMs while demonstrating strong generalization.

References

[1] Uri Alon and Eran Yahav. 2021. On the Bottleneck of Graph Neural Networks and its Practical Implications. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. https://openreview.net/forum?id=i80OPhOCVH2

[2] Amazon Web Services. 2016. EC2 Instances (F1) with Programmable Hardware. https://aws.amazon.com/blogs/aws/developer-preview-ec2-instances-f1-with-programmable-hardware/. Accessed: 2025-10-25.

[3] Anthropic. 2024. Model Card and Evaluations for Claude 3. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf. Accessed: 2026-04-03.

[4] Yunsheng Bai, Atefeh Sohrabizadeh, Zijian Ding, Rongjian Liang, Weikai Li, Ding Wang, Haoxing Ren, Yizhou Sun, and Jason Cong. 2024. Learning to Compare Hardware Designs for High-Level Synthesis. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD, MLCAD 2024, Salt Lake City, UT, USA, September 9-11, 2024*, Hussam Amrouch, Jiang Hu, Siddharth Garg, and Yibo Lin (Eds.). ACM, 2. https://doi.org/10.1145/3670474.3685940

[5] Yunsheng Bai, Atefeh Sohrabizadeh, Zongyue Qin, Ziniu Hu, Yizhou Sun, and Jason Cong. 2023. Towards a comprehensive benchmark for high-level synthesis targeted to fpgas. *Advances in Neural Information Processing Systems* 36 (2023), 45288–45299.

[6] Yunsheng Bai, Atefeh Sohrabizadeh, Yizhou Sun, and Jason Cong. 2022. Improving GNN-based accelerator design automation with meta learning. In *DAC '22: 59th ACM/IEEE Design Automation Conference, San Francisco, California, USA, July 10 - 14, 2022*, Rob Oshana (Ed.). ACM, 1347–1350. https://doi.org/10.1145/3489517.3530629

[7] Suhail Basalama and Jason Cong. 2025. Stream-HLS: Towards Automatic Dataflow Acceleration. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays, FPGA 2025, Monterey, CA, USA, 27 February 2025 - 1 March 2025*, Andrew Putnam and Jing Li (Eds.). ACM, 103–114. https://doi.org/10.1145/3706628.3708878

[8] Christophe Bobda, Joel Mandebi Mbongue, Paul Chow, Mohammad Ewais, Naif Tarafdar, Juan Camilo Vega, Ken Eguro, Dirk Koch, Suranga Handagala, Miriam Leeser, et al. 2022. The future of FPGA acceleration in datacenters and the cloud. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 15, 3 (2022), 1–42.

[9] Vinay Chamola, Sambit Patra, Neeraj Kumar, and Mohsen Guizani. 2020. FPGA for 5G: Re-configurable hardware for next generation communication. *IEEE Wireless Communications* 27, 3 (2020), 140–147.

[10] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.

[11] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. 2024. Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024 (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR / OpenReview.net, 6621–6642. https://proceedings.mlr.press/v235/chen24j.html

[12] Yuze Chi, Weikang Qiao, Atefeh Sohrabizadeh, Jie Wang, and Jason Cong. 2022. Democratizing domain-specific computing. *Commun. ACM* 66, 1 (2022), 74–85.

[13] Luca Collini, Siddharth Garg, and Ramesh Karri. 2025. C2hlsc: Leveraging large language models to bridge the software-to-hardware design gap. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–24.

[14] Jason Cong, Zhenman Fang, Michael Lo, Hanrui Wang, Jingxian Xu, and Shaochong Zhang. 2018. Understanding performance differences of FPGAs and GPUs. In *2018 IEEE 26th annual international symposium on field-programmable custom computing machines (FCCM)*. IEEE, 93–96.

[15] Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees Vissers, and Zhiru Zhang. 2011. High-level synthesis for FPGAs: From prototyping to deployment. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 30, 4 (2011), 473–491.

[16] Jason Cong, Bin Liu, Stephen Neuendorffer, Juanjo Noguera, Kees A. Vissers, and Zhiru Zhang. 2011. High-Level Synthesis for FPGAs: From Prototyping to Deployment. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 30, 4 (2011), 473–491. https://doi.org/10.1109/TCAD.2011.2110592

[17] Chris Cummins, Zacharias V. Fisches, Tal Ben-Nun, Torsten Hoefler, Michael F. P. O'Boyle, and Hugh Leather. 2021. ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event (Proceedings of Machine Learning Research)*, Marina Meila and Tong Zhang (Eds.). PMLR, 2244–2253. http://proceedings.mlr.press/v139/cummins21a.html

[18] Steve Dai, Yuan Zhou, Hang Zhang, Ecenur Ustun, Evangeline FY Young, and Zhiru Zhang. 2018. Fast and accurate estimation of quality of results in high-level synthesis with machine learning. In *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 129–132.

[19] Dimitrios Danopoulos, Konstantinos Anagnostopoulos, Christoforos Kachris, and Dimitrios Soudris. 2021. FPGA Acceleration of Generative Adversarial Networks for Image Reconstruction. In *2021 10th International Conference on Modern Circuits and Systems Technologies (MOCAST)*. IEEE, 1–5.

[20] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 2022. 8-bit Optimizers via Block-wise Quantization. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. https://openreview.net/forum?id=shpkpVXzo3h

[21] Aggelos Ferikoglou, Andreas Kakolyris, Vasilis Kypriotis, Dimosthenis Masouros, Dimitrios Soudris, and Sotirios Xydis. 2023. CollectiveHLS: Ultrafast Knowledge-Based HLS Design Optimization. *IEEE Embedded Systems Letters* 16, 2 (2023), 235–238.

[22] Aggelos Ferikoglou, Andreas Kakolyris, Vasilis Kypriotis, Dimosthenis Masouros, Dimitrios Soudris, and Sotirios Xydis. 2024. Data-driven HLS optimization for reconfigurable accelerators. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.

[23] Aggelos Ferikoglou, Andreas Kakolyris, Dimosthenis Masouros, Dimitrios Soudris, and Sotirios Xydis. 2024. CollectiveHLS: A collaborative approach to high-level synthesis design optimization. *ACM Transactions on Reconfigurable Technology and Systems* 18, 1 (2024), 1–32.

[24] Aggelos Ferikoglou, Despoina Tomkou, Dimosthenis Masouros, Dimitrios Soudris, and Sotirios Xydis. 2026. GNΩSIS: Lessons Learned in Generating a High-Level Synthesis Dataset. *ACM Transactions on Architecture and Code Optimization* (2026).

[25] Lorenzo Ferretti, Giovanni Ansaloni, and Laura Pozzi. 2018. Cluster-based heuristic for high level synthesis design space exploration. *IEEE Transactions on Emerging Topics in Computing* 9, 1 (2018), 35–43.

[26] Lorenzo Ferretti, Giovanni Ansaloni, and Laura Pozzi. 2018. Lattice-traversing design space exploration for high level synthesis. In *2018 IEEE 36th International Conference on Computer Design (ICCD)*. IEEE, 210–217.

[27] Lorenzo Ferretti, Andrea Cini, Georgios Zacharopoulos, Cesare Alippi, and Laura Pozzi. 2022. Graph neural networks for high-level synthesis design space exploration. *ACM Transactions on Design Automation of Electronic Systems* 28, 2 (2022), 1–20.

[28] Lorenzo Ferretti, Jihye Kwon, Giovanni Ansaloni, Giuseppe Di Guglielmo, Luca Carloni, and Laura Pozzi. 2021. Db4hls: A database of high-level synthesis design space explorations. *IEEE Embedded Systems Letters* 13, 4 (2021), 194–197.

[29] Lorenzo Ferretti, Jihye Kwon, Giovanni Ansaloni, Giuseppe Di Guglielmo, Luca P Carloni, and Laura Pozzi. 2020. Leveraging prior knowledge for effective design-space exploration in high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 3736–3747.

[30] Yonggan Fu, Yongan Zhang, Zhongzhi Yu, Sixu Li, Zhifan Ye, Chaojian Li, Cheng Wan, and Yingyan Celine Lin. 2023. Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.

[31] Mingzhe Gao, Jieru Zhao, Zhe Lin, and Minyi Guo. 2024. Hierarchical Source-to-Post-Route QoR Prediction in High-Level Synthesis with GNNs. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2024, Valencia, Spain, March 25-27, 2024*. IEEE, 1–6. https://doi.org/10.23919/DATE58400.2024.10546555

[32] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *CoRR* abs/2401.14196 (2024). https://doi.org/10.48550/ARXIV.2401.14196 arXiv:2401.14196

[33] Charles Hong, Sahil Bhatia, Altan Haan, Shengjun Kris Dong, Dima Nikiforov, Alvin Cheung, and Yakun Sophia Shao. 2024. Llm-aided compilation for tensor accelerators. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–14.

[34] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. https://openreview.net/forum?id=nZeVKeeFYf9

[35] Raisa Islam and Imtiaz Ahmed. 2024. Gemini-the most powerful LLM: Myth or Truth. In *2024 5th Information Communication Technologies Conference (ICTC)*. IEEE, 303–308.

[36] Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. 2019. CTRL: A Conditional Transformer Language Model for Controllable Generation. *CoRR* abs/1909.05858 (2019). arXiv:1909.05858 http://arxiv.org/abs/1909.05858

[37] Chris Lattner and Vikram S. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *2nd IEEE / ACM International Symposium on Code Generation and Optimization (CGO 2004), 20-24 March 2004, San Jose, CA, USA*. IEEE Computer Society, 75–88. https://doi.org/10.1109/CGO.2004.1281665

[38] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[39] Weikai Li, Ding Wang, Zijian Ding, Atefeh Sohrabizadeh, Zongyue Qin, Jason Cong, and Yizhou Sun. 2025. Hierarchical Mixture of Experts: Generalizable Learning for High-Level Synthesis. In *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, Toby Walsh, Julie Shah, and Zico Kolter (Eds.). AAAI Press, 18476–18484. https://doi.org/10.1609/AAAI.V39I17.34033

[40] Zhe Lin, Jieru Zhao, Sharad Sinha, and Wei Zhang. 2020. HL-Pow: A learning-based power modeling framework for high-level synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 574–580.

[41] Anushree Mahapatra and Benjamin Carrion Schafer. 2014. Machine-learning based simulated annealer method for high level synthesis design space exploration. In *Proceedings of the 2014 Electronic System Level Synthesis Conference (ESLsyn)*. IEEE, 1–6.

[42] Hosein Mohammadi Makrani, Hossein Sayadi, Tinoosh Mohsenin, Setareh Rafatirad, Avesta Sasan, and Houman Homayoun. 2019. Xppe: cross-platform performance estimation of hardware accelerators using machine learning. In *Proceedings of the 24th Asia and South Pacific Design Automation Conference*. 727–732.

[43] Dimosthenis Masouros, Aggelos Ferikoglou, Georgios Zervakis, Sotirios Xydis, and Dimitrios Soudris. 2024. Late Breaking Results: Language-level QoR modeling for High-Level Synthesis. In *Proceedings of the 61st ACM/IEEE Design Automation Conference, DAC 2024, San Francisco, CA, USA, June 23-27, 2024*, Vivek De (Ed.). ACM, 351:1–351:2. https://doi.org/10.1145/3649329.3663500

[44] Emmet Murphy and Lana Josipovic. 2024. Balor: HLS Source Code Evaluator Based on Custom Graphs and Hierarchical GNNs. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, ICCAD 2024, Newark Liberty International Airport Marriott, NJ, USA, October 27-31, 2024*, Jinjun Xiong and Robert Wille (Eds.). ACM, 227:1–227:9. https://doi.org/10.1145/3676536.3676788

[45] Mostafa W Numan, Braden J Phillips, Gavin S Puddy, and Katrina Falkner. 2020. Towards automatic high-level code deployment on reconfigurable platforms: A survey of high-level synthesis tools and toolchains. *IEEE Access* 8 (2020), 174692–174722.

[46] Kenta Oono and Taiji Suzuki. 2020. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net. https://openreview.net/forum?id=S1ldO2EFPr

[47] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). https://doi.org/10.48550/ARXIV.2303.08774 arXiv:2303.08774

[48] Jingyu Pan, Guanglei Zhou, Chen-Chia Chang, Isaac Jacobson, Jiang Hu, and Yiran Chen. 2025. A survey of research in large language models for electronic design automation. *ACM Transactions on Design Automation of Electronic Systems* 30, 3 (2025), 1–21.

[49] Alexandros Papakonstantinou, Yun Liang, John A Stratton, Karthik Gururaj, Deming Chen, Wen-Mei W Hwu, and Jason Cong. 2011. Multilevel granularity parallelism synthesis on FPGAs. In *2011 IEEE 19th Annual International Symposium on Field-Programmable Custom Computing Machines*. IEEE, 178–185.

[50] Neha Prakriya, Zijian Ding, Yizhou Sun, and Jason Cong. 2025. LIFT: LLM-Based Pragma Insertion for HLS via GNN Supervised Fine-Tuning. *CoRR* abs/2504.21187 (2025). https://doi.org/10.48550/ARXIV.2504.21187 arXiv:2504.21187

[51] Andrew Putnam, Adrian M Caulfield, Eric S Chung, Derek Chiou, Kypros Constantinides, John Demme, Hadi Esmaeilzadeh, Jeremy Fowers, Gopi Prashanth Gopal, Jan Gray, et al. 2014. A reconfigurable fabric for accelerating large-scale datacenter services. *ACM SIGARCH Computer Architecture News* 42, 3 (2014), 13–24.

[52] Zongyue Qin, Yunsheng Bai, Atefeh Sohrabizadeh, Zijian Ding, Ziniu Hu, Yizhou Sun, and Jason Cong. 2024. Cross-Modality Program Representation Learning for Electronic Design Automation with High-Level Synthesis. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD, MLCAD 2024, Salt Lake City, UT, USA, September 9-11, 2024*, Hussam Amrouch, Jiang Hu, Siddharth Garg, and Yibo Lin (Eds.). ACM, 14. https://doi.org/10.1145/3670474.3685952

[53] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract-Conference.html

[54] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *CoRR* abs/1910.10683 (2019). arXiv:1910.10683 http://arxiv.org/abs/1910.10683

[55] Kyle Rupnow, Yun Liang, Yinan Li, and Deming Chen. 2011. A study of high-level synthesis: Promises and challenges. In *2011 9th IEEE International Conference on ASIC*. IEEE, 1102–1105.

[56] Benjamin Carrion Schafer and Zi Wang. 2019. High-level synthesis design space exploration: Past, present, and future. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 10 (2019), 2628–2639.

[57] Anirban Sengupta and Saumya Bhadauria. 2015. User power-delay budget driven PSO based design space exploration of optimal k-cycle transient fault secured datapath during high level synthesis. In *Sixteenth International Symposium on Quality Electronic Design*. IEEE, 289–292.

[58] Atefeh Sohrabizadeh, Yunsheng Bai, Yizhou Sun, and Jason Cong. 2022. Automated Accelerator Optimization Aided by Graph Neural Networks. In *FPGA '22: The 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, Virtual Event, USA, 27 February 2022 - 1 March 2022*, Michael Adler and Paolo Ienne (Eds.). ACM, 50. https://doi.org/10.1145/3490422.3502330

[59] Atefeh Sohrabizadeh, Yunsheng Bai, Yizhou Sun, and Jason Cong. 2023. Robust GNN-Based Representation Learning for HLS. In *IEEE/ACM International Conference on Computer Aided Design, ICCAD 2023, San Francisco, CA, USA, October 28 - Nov. 2, 2023*. IEEE, 1–9. https://doi.org/10.1109/ICCAD57390.2023.10323853

[60] Sneha Swaroopa, Rijoy Mukherjee, Anushka Debnath, and Rajat Subhra Chakraborty. 2025. Evaluating Large Language Models for Automatic Register Transfer Logic Generation for Combinational Circuits via High-Level Synthesis. *Foundatiosn and Trends in Electronic Design Automation* 14, 4 (2025), 295–314.

[61] Ecenur Ustun, Chenhui Deng, Debjit Pal, Zhijing Li, and Zhiru Zhang. 2020. Accurate operation delay prediction for FPGA HLS using graph neural networks. In *Proceedings of the 39th international conference on computer-aided design*. 1–9.

[62] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[63] vLLM Team. 2024. *vLLM Documentation: Dynamically Serving LoRA Adapters.* https://docs.vllm.ai/en/stable/features/lora/#dynamically-serving-lora-adapters Accessed: 2024-05-22.

[64] Zishen Wan, Bo Yu, Thomas Yuang Li, Jie Tang, Yuhao Zhu, Yu Wang, Arijit Raychowdhury, and Shaoshan Liu. 2021. A survey of fpga-based robotic computing. *IEEE Circuits and Systems Magazine* 21, 2 (2021), 48–74.

[65] Hanyu Wang, Xinrui Wu, Zijian Ding, Su Zheng, Chengyue Wang, Neha Prakriya, Tony Nowatzki, Yizhou Sun, and Jason Cong. 2025. LLM-DSE: Searching Accelerator Parameters with LLM Agents. *arXiv preprint arXiv:2505.12188* (2025).

[66] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, 8696–8708. https://doi.org/10.18653/V1/2021.EMNLP-MAIN.685

[67] Nan Wu, Hang Yang, Yuan Xie, Pan Li, and Cong Hao. 2022. High-level synthesis performance prediction using GNNs: benchmarking, modeling, and advancing. In *DAC '22: 59th ACM/IEEE Design Automation Conference, San Francisco, California, USA, July 10 - 14, 2022*, Rob Oshana (Ed.). ACM, 49–54. https://doi.org/10.1145/3489517.3530408

[68] Chenwei Xiong, Cheng Liu, Huawei Li, and Xiaowei Li. 2024. HLSPilot: LLM-based High-Level Synthesis. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, ICCAD 2024, Newark Liberty International Airport Marriott, NJ, USA, October 27-31, 2024*, Jinjun Xiong and Robert Wille (Eds.). ACM, 226:1–226:9. https://doi.org/10.1145/3676536.3676781

[69] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net. https://openreview.net/forum?id=ryGs6iA5Km

[70] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation Learning on Graphs with Jumping Knowledge Networks. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018 (Proceedings of Machine Learning Research)*, Jennifer G. Dy and Andreas Krause (Eds.). PMLR, 5449–5458. http://proceedings.mlr.press/v80/xu18c.html

[71] Sotirios Xydis, Gianluca Palermo, Vittorio Zaccaria, and Cristina Silvano. 2014. SPIRIT: Spectral-aware Pareto iterative refinement optimization for supervised high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 34, 1 (2014), 155–159.

[72] Sotirios Xydis, Kiamal Pekmestzi, Dimitrios Soudris, and George Economakos. 2013. Compiler-in-the-loop exploration during datapath synthesis for higher quality delay-area trade-offs. *ACM Trans. Des. Autom. Electron. Syst.* 18, 1, Article

11 (Jan. 2013), 35 pages. https://doi.org/10.1145/2390191.2390202

[73] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. 2022. Scalehls: A new scalable high-level synthesis framework on multi-level intermediate representation. In *2022 IEEE international symposium on high-performance computer architecture (HPCA)*. IEEE, 741–755.